UNIVERSITÉ DE MONTPELLIER

Master Informatique Parcours ICO

Compte rendu du TP 1 Java RMI

Cabinet vétérinaire distribué

HAI704I Architectures logicielles distribuées

Étudiant	Omar RHANBOURI
Numéro étudiant	22204411
Enseignant responsable	Bachar RIMA
Année universitaire	2026/2027
Date	27 septembre 2026
Périmètre	Partie A et partie C bonus Swing

Dépôt GitHub : ***https://github.com/SJOLK/RHANBOURI_Omar_HAI704I_TP1***

Vidéo de démonstration : ***https://www.youtube.com/watch?v=eG6Rcq-QFNw***

Table des matières

1 Objectifs et périmètre 3

2 Architecture distribuée 4

3 Étapes A0 et A1 5

4 Passage par valeur et référence 6

5 Cabinet et création des patients 8

6 Callbacks et alertes 9

7 Expérience de sérialisation A7 10

8 Interface en ligne de commande A8 12

9 Validation et comportement face aux pannes 13

10 Interface graphique Swing partie C 14

11 Synthèse et regard critique 17

12 Usage des outils d intelligence artificielle générative 18

13 Références 18

14 Captures de validation 19

1 Objectifs et périmètre

Ce TP met en œuvre une application distribuée en Java RMI natif. Le serveur héberge un cabinet vétérinaire et les objets métier partagés. Les clients disposent d'une interface en ligne de commande et d'une interface graphique Swing qui réutilisent la même logique cliente. Le résultat respecte les trois modules common, server et client et ne dépend d'aucune bibliothèque externe.

La partie A est complète et la partie C bonus ajoute une interface graphique utilisable. Le registre publie uniquement le cabinet sous le nom Cabinet. Les animaux et leurs dossiers restent dans la JVM du serveur et sont accessibles par référence distante, tandis que l'espèce traverse le réseau par valeur. Les clients peuvent exporter un observateur afin que le serveur les rappelle lors du franchissement des seuils de patients.

1 1 Résultats obtenus

Étape	Choix mis en œuvre	Validation
A0	Trois modules non modulaires et classpaths séparés	Compilation et exécution en processus séparés
A1	Animal exporté comme objet distant	Stub dynamique et Proxy.isProxyClass à true
A2	Espèce sérialisable transmise par valeur	Copies distinctes et modification locale isolée
A3	Dossier de suivi distant	Modification visible depuis un autre client
A4	Cabinet comme point d entrée unique	Une seule liaison Cabinet dans le registre
A5	Création serveur déclenchée par le client	Nombre de patients augmenté et recherche réussie
A6	Callbacks distribués	Alerte reçue par deux clients au seuil 100
A7	Expérience de sérialisation	Échec expliqué puis corrigé par partage de la classe
A8	CLI interactive séparée de RMI	Menu complet, erreurs gérées et arrêt propre
C	Interface graphique Swing réutilisant LogiqueClient	Opérations principales, SwingWorker et callback affiché dans la fenêtre

1 2 Environnement

- JDK Microsoft OpenJDK 21.0.6, compatible avec le minimum Java 17 demandé.
- IntelliJ IDEA 2025.3 pour la compilation et le lancement de plusieurs processus.
- Registre RMI créé dans la JVM du serveur sur localhost et le port 1099.
- Dépôt Git contenant les sources et le README, sans classes compilées ni fichiers IntelliJ suivis.

2 Architecture distribuée

L'architecture sépare le contrat, l'implémentation serveur et l'interface cliente. Le module common est la seule dépendance partagée. Le client ne référence aucune classe du module server et ne connaît que les interfaces distantes ainsi que les types sérialisables.

Architecture de déploiement et dépendances

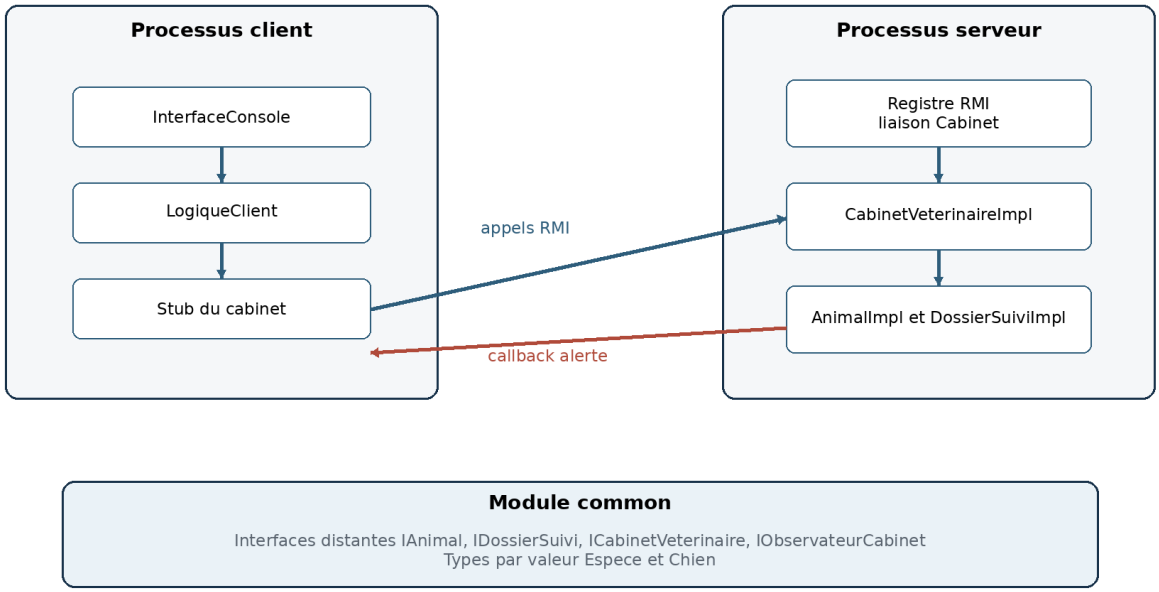


Figure 1 Composants déployés et sens des appels distants

2 1 Rôle des modules

Module	Responsabilité	Dépendance
common	Contrats distants et types transmis par valeur	server et client
server	Servants, état métier et publication dans le registre	common
client	Lookup, logique cliente, interfaces console et Swing, observateur exporté	common

2 2 Déploiement et registre

Le serveur crée le registre avec LocateRegistry.createRegistry sur le port 1099, construit le cabinet puis exécute rebind avec le nom Cabinet. Le client récupère le registre, effectue lookup et reçoit un stub. Le registre sert donc uniquement à amorcer la communication. Les animaux et dossiers sont ensuite atteints en naviguant depuis le cabinet.

3 Étapes A0 et A1

3 1 Mise en route A0

J'ai d'abord vérifié l'environnement Java puis repris la structure multi-projets fournie. Les modules client et server placent common sur leur classpath. Aucun module-info.java n'est utilisé. Cette organisation rend explicite la frontière de compilation et empêche le client de dépendre directement des implémentations du serveur.

3 2 Premier animal distant A1

Le contrat IAnimal étend java.rmi.Remote et chacune de ses opérations déclare RemoteException. AnimalImpl est instancié et exporté côté serveur. Lors du lookup, le client ne reçoit pas l'objet réel mais un proxy dynamique qui transmet les appels au processus serveur.

Observation	Interprétation
Classe jdk.proxy1.$Proxy2	Le client manipule un stub engendré à l exécution.
Proxy.isProxyClass renvoie true	L appel traverse bien la couche RMI.
Nom de Rex affiché dans le client	La méthode du servant est invoquée à distance.

3 3 Risques évités

- Le client ne construit pas AnimalImpl et ne référence pas le module server.
- Le contrat partagé ne contient que des types connus des deux JVM.
- Les erreurs réseau restent visibles dans les signatures grâce à RemoteException.

4 Passage par valeur et référence

Le projet utilise volontairement les deux régimes de passage de RMI. Le choix dépend de l'identité de l'objet, du partage attendu entre clients et de l'endroit où les modifications doivent être conservées.

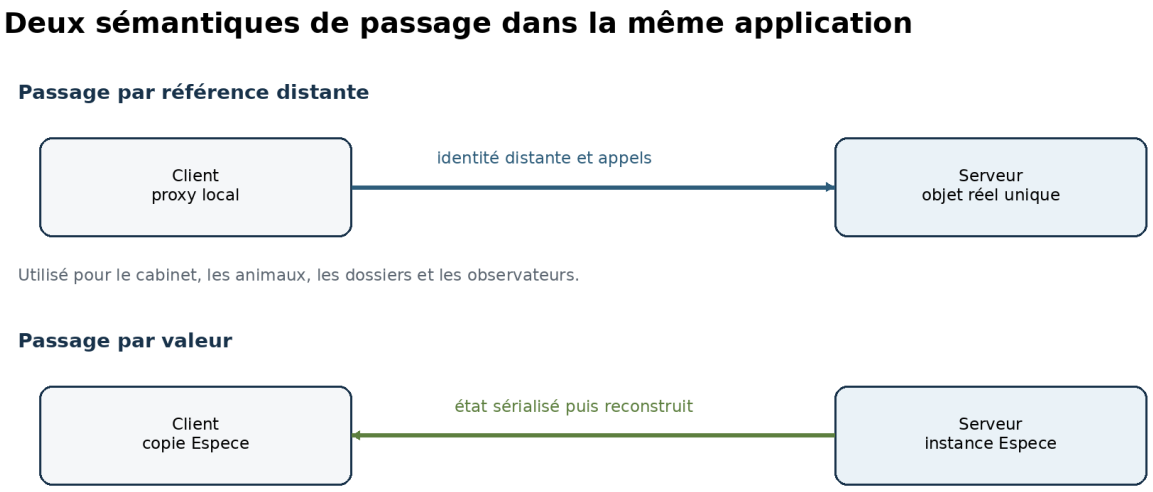


Figure 2 Référence distante et copie sérialisée

4 1 Espèce transmise par valeur A2

Espece est une donnée descriptive composée d'un nom et d'une espérance de vie moyenne. Elle implémente Serializable et réside dans common afin que le client et le serveur puissent charger sa classe. Lorsqu'une méthode distante renvoie une espèce, RMI sérialise son état puis reconstruit une nouvelle instance dans la JVM cliente.

Le test modifie localement la première copie reçue de Chien 13 ans vers Chat 20 ans. Une nouvelle récupération renvoie encore Chien 13 ans. Les codes produits par System.identityHashCode diffèrent également entre les récupérations. Ces observations montrent que l'espèce est une copie sans identité distante partagée.

4 2 Dossier distant A3

Le dossier de suivi doit être partagé par plusieurs vétérinaires. Je l'ai donc modélisé comme objet distant. DossierSuiviImpl reste côté serveur et le client reçoit une référence IDossierSuivi. Une modification de l'état de santé ou de l'historique s'applique à l'unique dossier associé au patient.

Ce choix est validé par un test à deux clients. Le premier place le dossier de alfa dans l'état en observation et ajoute l'observation Modification effectuée depuis le client 1. Le second récupère ensuite le même dossier et voit l'état ainsi que l'observation. Une copie sérialisée n'aurait pas fourni cette cohérence sans opération explicite de remplacement.

4 3 Synthèse des choix de passage

Objet	Régime	Raison
Cabinet	Référence distante	Point d entrée unique et état serveur partagé
Animal	Référence distante	Patient conservé et recherché côté serveur
Dossier de suivi	Référence distante	Modifications visibles par tous les clients
Observateur client	Référence distante	Le serveur doit rappeler la JVM cliente
Espèce	Valeur	Donnée descriptive consultée comme une copie

4 4 Conséquence architecturale

La règle suivie est de conserver par référence les objets dont l'identité ou les modifications doivent être partagées. Le cabinet, les animaux et les dossiers restent ainsi dans la JVM du serveur. À l'inverse, une espèce décrit un patient sans porter l'état partagé du cabinet, ce qui rend la copie par valeur adaptée.

L'observateur constitue le cas symétrique. Son objet réel réside dans la JVM cliente et le serveur conserve son stub. Un appel RMI peut donc circuler dans les deux sens sans déplacer les objets qui portent l'identité distribuée.

5 Cabinet et création des patients

5 1 Point d entrée unique A4

ICabinetVeterinaire fournit les opérations nécessaires pour lister les patients et rechercher un patient par son nom. Le serveur ne publie plus les animaux individuellement. La commande de diagnostic du registre affiche uniquement la liaison Cabinet.

La liste renvoyée contient des références IAnimal. Le conteneur de liste traverse le réseau, mais chaque élément désigne un animal distant. Une recherche infructueuse est transformée côté client en résultat métier clair, affiché comme introuvable, sans arrêter la CLI.

5 2 Création déclenchée depuis le client A5

Le client transmet les données nécessaires à la création, dont une instance Espece par valeur. Le cabinet reçoit ces informations puis instancie AnimalImpl et DossierSuiviImpl dans la JVM du serveur. Le nouvel animal est ainsi immédiatement un objet distant valide et son état reste dans la collection du cabinet.

1. Le client récupère le nombre de patients avant l'appel.
2. Il transmet le nom, le maître, la race et l'espèce sérialisée.
3. Le serveur construit et exporte l'animal ainsi que son dossier.
4. Le client vérifie l'augmentation du nombre puis retrouve le patient par son nom.

Validation	Résultat observé
Ajout de Rocky	Le nombre passe de 2 à 3 et la recherche retrouve Rocky.
Ajout de alfa depuis la CLI	alfa apparaît dans la liste avec son espèce et son dossier.
Objet retourné	Le patient reçu est un proxy RMI.

6 Callbacks et alertes

6 1 Modèle Observer distribué A6

IObservateurCabinet est défini dans common. Chaque client construit ObservateurCabinetImpl puis l'exporte. L'appel d'abonnement transmet au serveur le stub de cet observateur, pas son implémentation. Le client devient donc lui-même hébergeur d'un objet distant que le serveur peut invoquer.

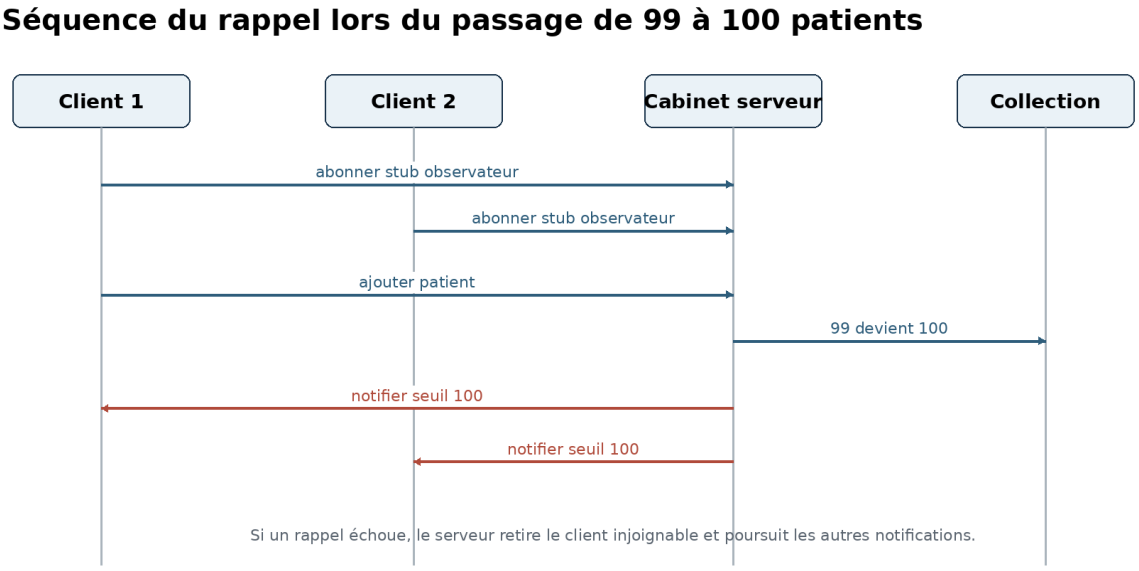


Figure 3 Abonnement et notification de deux clients

6 2 Détection des franchissements

Le cabinet compare l'effectif avant et après chaque modification. Une alerte est envoyée seulement si une opération traverse un seuil de 100, 500 ou 1000 patients, à la hausse ou à la baisse. Cette comparaison évite de notifier de nouveau les clients lorsqu'une modification reste du même côté du seuil.

6 3 Robustesse des rappels

Chaque notification est un appel distant et peut échouer si le client a été fermé. Le serveur traite les abonnés séparément. L'échec d'un rappel ne doit ni interrompre l'ajout du patient ni empêcher la notification des autres clients. L'observateur devenu injoignable peut être retiré de la collection pour éviter des échecs répétés.

Le test réalisé avec deux clients abonnés a provoqué une alerte dans les deux consoles lors du passage de 99 à 100 patients. L'alerte indique le nom du client, le seuil franchi et l'évolution 99 vers 100.

7 Expérience de sérialisation A7

7 1 Protocole

J'ai défini une sous-classe Chien de Espece uniquement dans le projet client, puis j'ai envoyé une instance au serveur via l'opération d'ajout de patient. L'appel compile, car le client connaît Chien et la signature distante accepte une Espece. L'échec apparaît au moment où le serveur doit désérialiser l'argument.

7 2 Échec observé

La cause profonde est java.lang.ClassNotFoundException pour animal.client.Chien. RMI l'encapsule dans UnmarshalException, elle-même renvoyée au client par ServerException. La chaîne d'exceptions situe correctement le problème dans le décodage des arguments du côté serveur.

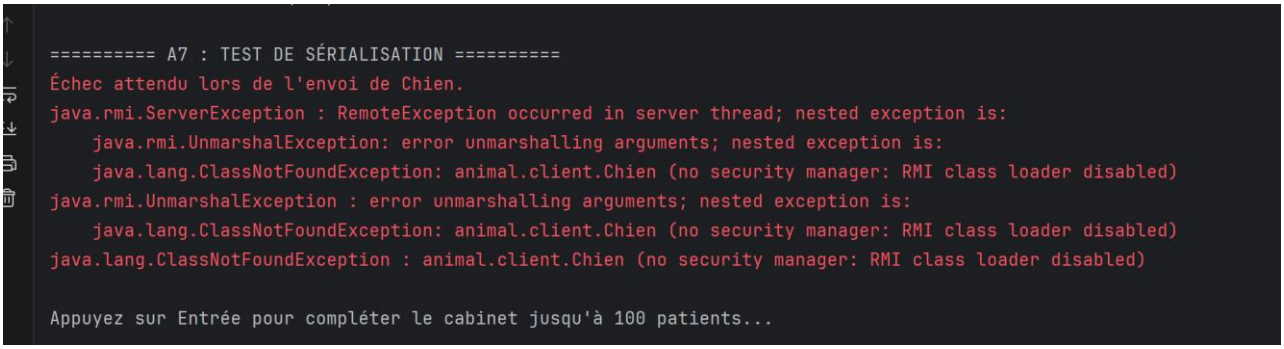


Figure 4 Échec attendu lorsque Chien existe uniquement dans client

7 3 Explication

Un flux de sérialisation contient le nom qualifié de la classe, les métadonnées nécessaires au graphe d'objets et les valeurs des champs sérialisés. Il ne transporte pas le bytecode de la classe. Le serveur doit donc déjà disposer de la définition animal.client.Chien sur son classpath. Comme le module server ne dépend pas de client, le chargeur de classes ne peut pas reconstruire l'objet.

Cette expérience montre aussi pourquoi les types transmis par valeur appartiennent à common. Le contrat Java ne se limite pas aux interfaces distantes. Il inclut également toutes les classes concrètes susceptibles d'apparaître dans un flux sérialisé.

7 4 Correction

La correction la plus simple consiste à déplacer Chien dans common, dans un paquetage commun, puis à recompiler common, server et client. Les deux JVM chargent alors exactement la même classe au moment de la sérialisation et de la désérialisation.

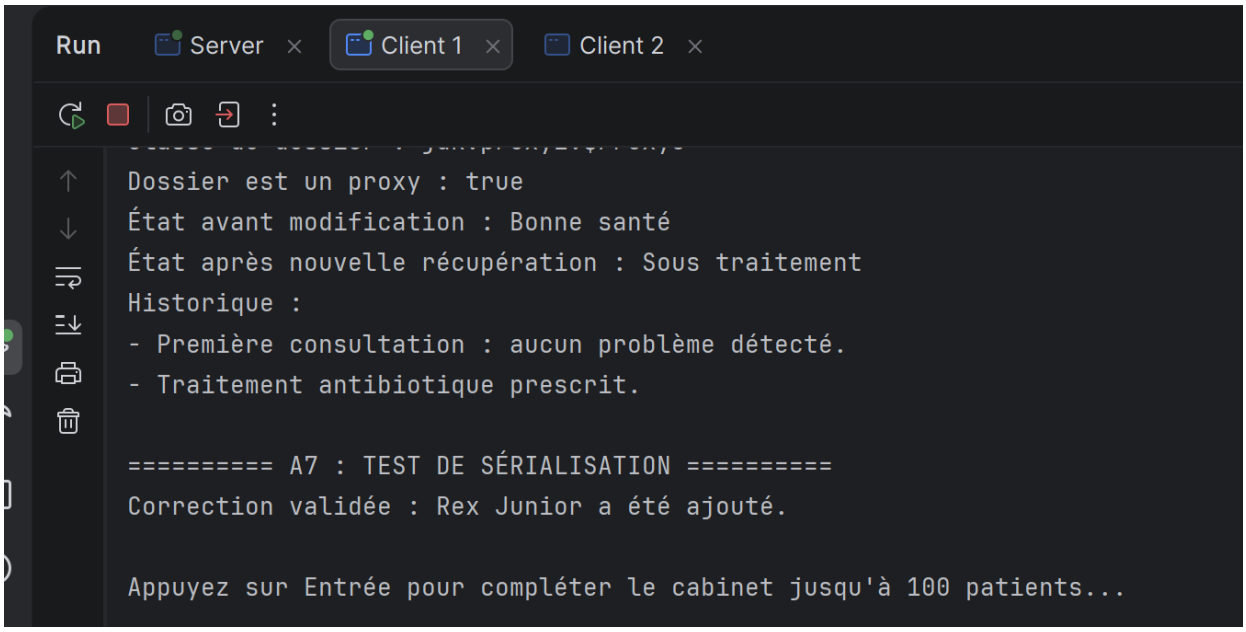


Figure 5 Ajout réussi après déplacement de Chien dans common

État	Classpath serveur	Résultat
Avant correction	Espece connue mais Chien absent	ClassNotFoundException pendant unmarshal
Après correction	Espece et Chien disponibles dans common	Rex Junior est ajouté normalement

7 5 Enseignement

L'héritage ne permet pas au serveur de reconstruire automatiquement une sous-classe inconnue. La classe réelle de l'objet sérialisé compte, même lorsque le paramètre est déclaré avec un type parent. Le classpath est donc une partie du protocole de fait entre les processus.

8 Interface en ligne de commande A8

8 1 Séparation des responsabilités

Composant	Responsabilité	Contrainte
Client	Lire hôte et port, effectuer le lookup, lancer et fermer le client	Point d entrée unique
LogiqueClient	Regrouper les appels distants et convertir les erreurs	Dépend des interfaces common
InterfaceConsole	Afficher le menu, lire et valider les saisies	Aucun appel direct à java.rmi
ObservateurCabinetImpl	Afficher les alertes reçues du serveur	Objet exporté par la JVM cliente
ErreurClientException	Présenter une erreur compréhensible à l interface	Évite les traces techniques dans le menu

Cette organisation isole l'interaction utilisateur de la communication distante. InterfaceConsole et InterfaceGraphique appellent uniquement LogiqueClient. La logique de lookup, les opérations métier, la gestion des abonnements et le traitement des RemoteException ne sont donc pas dupliqués entre les deux interfaces.

8 2 Fonctionnalités du menu

- Afficher tous les patients et rechercher un patient par son nom.
- Enregistrer un patient avec son maître, sa race et son espèce.
- Consulter puis modifier l'état et l'historique du dossier de suivi.
- S'abonner aux alertes ou se désabonner pendant l'utilisation.
- Quitter proprement en libérant l'observateur exporté.

8 3 Saisies et pannes

Une valeur de menu inconnue, par exemple 99, produit le message Choix invalide. Veuillez réessayer et la boucle continue. Les valeurs numériques sont lues avec validation afin qu'une saisie incorrecte ne provoque pas la fin du processus.

Les exceptions RMI sont traduites en messages courts. Lorsque le serveur devient inaccessible, l'interface affiche Impossible de communiquer avec le serveur. La fermeture reste possible et indique que le serveur est inaccessible pendant le nettoyage. Le processus client termine malgré cette panne.

9 Validation et comportement face aux pannes

9 1 Scénario multi clients

1. Démarrer le serveur et vérifier la publication de Cabinet.
2. Lancer deux processus clients séparés sur localhost et le port 1099.
3. Ajouter alfa depuis le premier client puis vérifier sa présence.
4. Modifier son dossier depuis le premier client.
5. Consulter le même dossier depuis le second client et observer la modification.
6. Abonner les deux clients et provoquer le franchissement du seuil 100.
7. Fermer les clients avec le choix 0 et vérifier le code de sortie 0.

9 2 Matrice de validation

Test	Observation	État
Liste initiale	Rex et Mimi affichés	Conforme
Recherche absente	Message introuvable	Conforme
Ajout	alfa apparaît et possède un dossier	Conforme
Partage dossier	État en observation visible depuis client 2	Conforme
Saisie 99	Message de choix invalide, menu maintenu	Conforme
Alerte seuil 100	Affichage dans les deux consoles	Conforme
Serveur arrêté	Message compréhensible, client encore contrôlable	Conforme
Quitter	Désabonnement ou nettoyage puis code 0	Conforme

9 3 Limites du protocole de test

Les validations sont des tests manuels reproductibles à partir de la CLI et des consoles séparées. Elles couvrent les comportements demandés, mais elles ne remplacent pas des tests automatisés de concurrence, de charge ou de reprise après redémarrage. L'état est conservé en mémoire et disparaît lorsque le serveur s'arrête.

10 Interface graphique Swing partie C

La partie C propose une interface graphique Swing connectée au même serveur RMI que le client console. Elle constitue une nouvelle couche de présentation et conserve les contrats distants, les objets serveur et la logique cliente déjà validés dans la partie A.

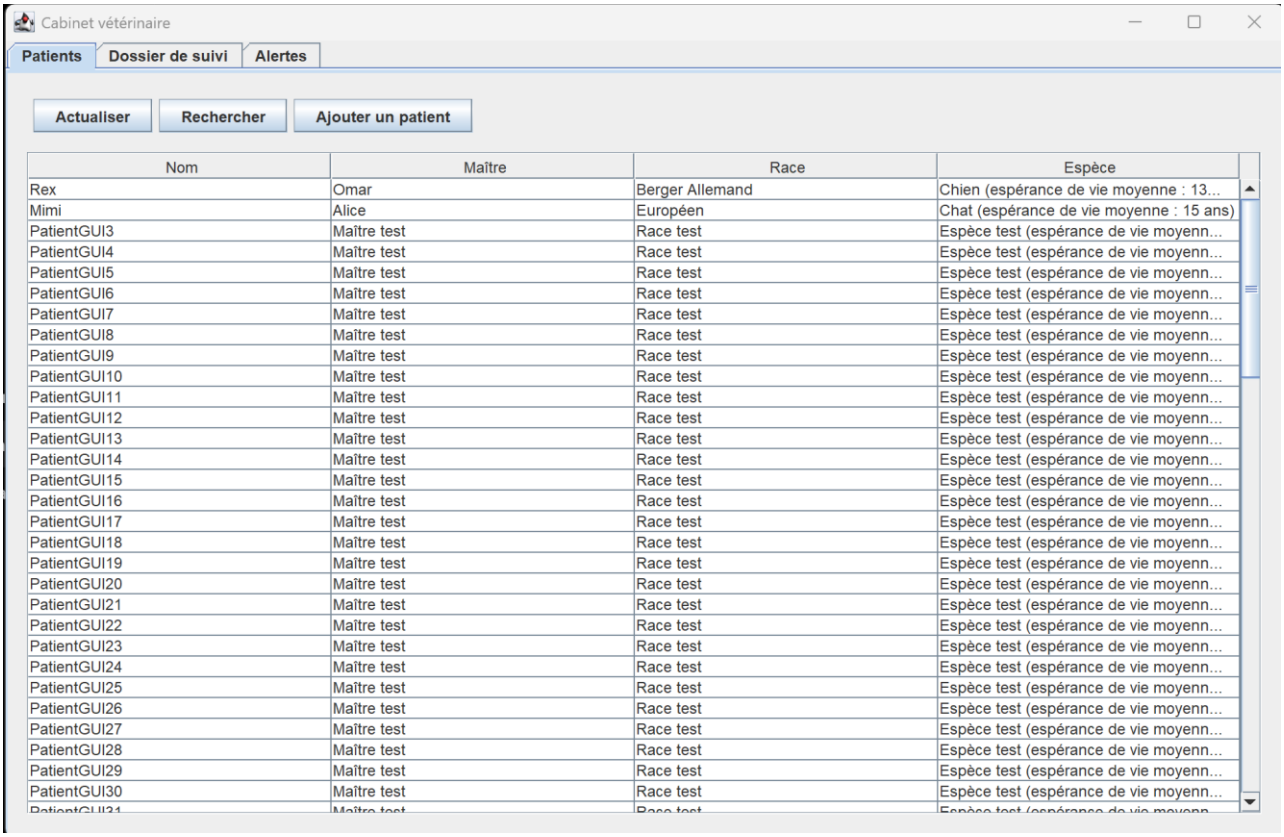
10 1 Réutilisation de la logique cliente

ClientGraphique réalise le lookup du cabinet puis construit LogiqueClient, comme le point d'entrée console. InterfaceGraphique utilise ensuite cette instance pour lister, rechercher et ajouter les patients, consulter ou modifier les dossiers et gérer les abonnements. Aucun composant Swing n'appelle directement les stubs RMI.

Onglet	Fonctions disponibles	Méthodes de LogiqueClient réutilisées
Patients	Lister, rechercher et ajouter un patient	listerPatients, rechercherPatient, ajouterPatient
Dossier de suivi	Consulter l'état, lire l'historique et modifier le dossier	consulterDossier, mettreAJourDossier
Alertes	S'abonner, se désabonner et afficher les callbacks	abonner, desabonner, estAbonne

10 2 Interface utilisable

L'onglet Patients présente les données dans un tableau et propose les opérations principales sous forme de boutons. Les formulaires contrôlent les valeurs obligatoires et les erreurs métier ou réseau sont affichées dans des boîtes de dialogue compréhensibles.



Nom	Maître	Race	Espèce
Rex	Omar	Berger Allemand	Chien (espérance de vie moyenne : 13...
Mimi	Alice	Européen	Chat (espérance de vie moyenne : 15 ans)
PatientGUI3	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI4	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI5	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI6	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI7	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI8	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI9	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI10	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI11	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI12	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI13	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI14	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI15	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI16	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI17	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI18	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI19	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI20	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI21	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI22	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI23	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI24	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI25	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI26	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI27	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI28	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI29	Maître test	Race test	Espèce test (espérance de vie moyenn...
PatientGUI30	Maître test	Race test	Espèce test (espérance de vie moyenn...

Figure C1 Liste des patients dans l'interface Swing

10 3 Appels asynchrones et callbacks

Les appels RMI sont exécutés dans des SwingWorker afin de ne pas bloquer l'Event Dispatch Thread. Lorsque le serveur appelle l'observateur, le callback arrive sur un thread RMI. La mise à jour de la zone d'alertes est alors programmée avec SwingUtilities.invokeLater pour respecter le modèle de concurrence de Swing.

ObservateurCabinetImpl accepte un Consumer de chaînes. Le constructeur historique conserve l'affichage console, tandis que l'interface graphique fournit un consommateur qui ajoute le message dans la zone Swing. Cette surcharge maintient la compatibilité du client console et évite une seconde implémentation de l'observateur.

10 4 Validation du seuil 100

Le bouton Test seuil 100 complète le cabinet en arrière-plan. Le passage de 99 à 100 déclenche un véritable callback RMI. La notification indique le client concerné, le seuil franchi et l'évolution du nombre de patients. La fermeture de la fenêtre appelle LogiqueClient.fermer afin de désabonner et désenregistrer proprement l'observateur exporté.

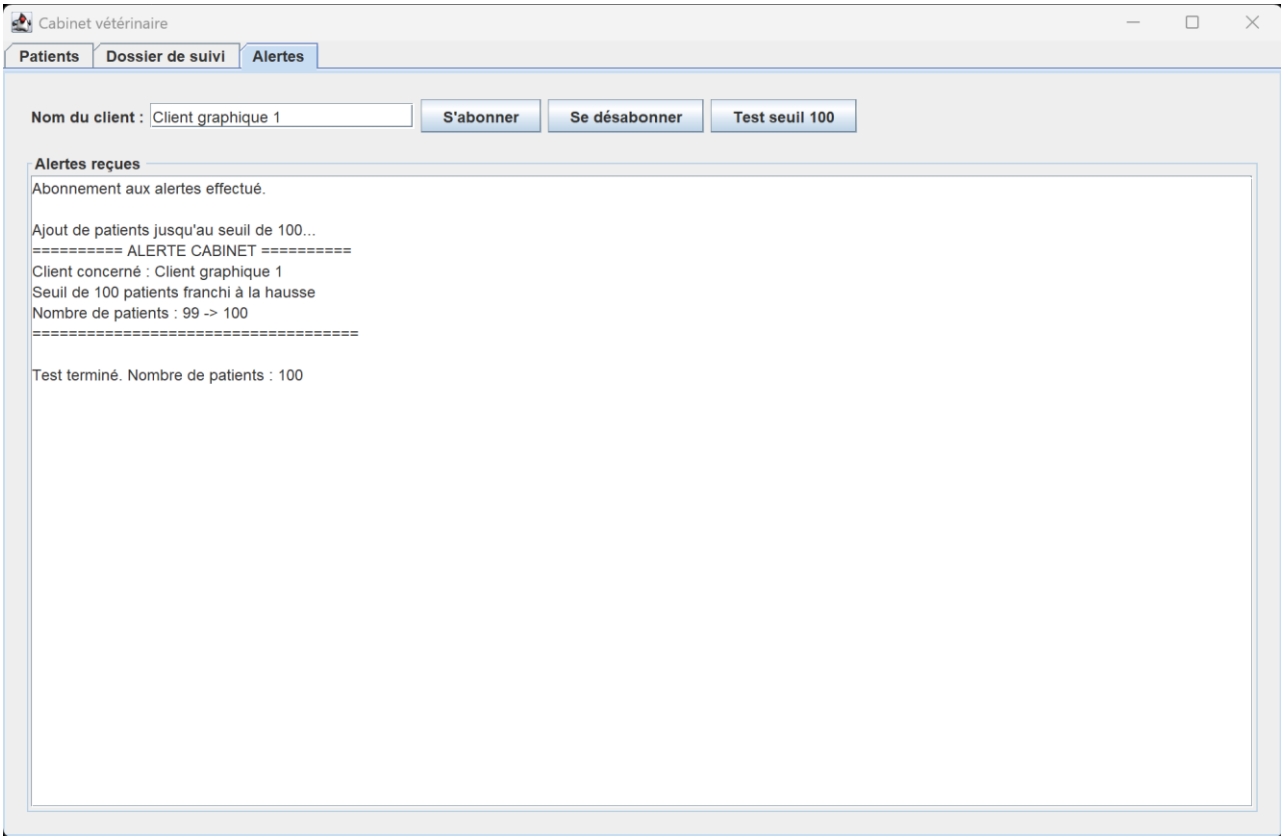


Figure C2 Callback RMI reçu et affiché dans l'interface Swing

Cette validation couvre les deux critères de la partie C : une interface graphique utilisable pour les opérations principales et la réutilisation effective de la logique du client console.

11 Synthèse et regard critique

11 1 Ce qui fonctionne

La partie A fournit une application RMI utilisable en processus séparés. Le cabinet reste l'unique objet publié, les patients et dossiers conservent une identité distante, les espèces sont copiées et les callbacks préviennent plusieurs clients. La CLI et l'interface Swing couvrent les opérations principales en réutilisant LogiqueClient. Les échecs prévisibles sont présentés sans interrompre brutalement l'interface active.

11 2 Difficultés rencontrées et résolution

Difficulté	Diagnostic	Résolution
Sous classe Chien refusée	ClassNotFoundException dans la chaîne RMI	Déplacement de Chien vers common
Deuxième client lancé avec Client 2 en argument	L argument était interprété comme un hôte	Lancer sans argument ou avec localhost 1099
Données absentes après redémarrage	La collection serveur est uniquement en mémoire	Recréer le patient après chaque redémarrage
Client abonné qui doit terminer	Un objet exporté peut maintenir la JVM active	Désabonnement et unexport lors de la fermeture

11 3 Améliorations possibles

- Ajouter une persistance locale ou une base de données pour conserver les patients après redémarrage.
- Protéger les collections partagées et ajouter des tests de concurrence avec plusieurs ajouts simultanés.
- Automatiser les tests d'intégration en lançant le registre, le serveur et plusieurs clients de test.
- Structurer les erreurs avec des exceptions métier distinctes pour les cas patient absent ou saisie invalide.
- Ajouter une journalisation configurable afin de séparer les traces techniques des messages destinés au vétérinaire.

La migration vers Spring Boot n'a pas été retenue. En revanche, la partie C bonus a été réalisée avec Swing afin de proposer une seconde présentation sans modifier les contrats RMI ni dupliquer la logique cliente.

12 Usage des outils d intelligence artificielle générative

J'ai utilisé ChatGPT d'OpenAI comme assistant pendant ce TP. Les sollicitations ont porté sur l'explication de concepts RMI, l'interprétation de messages d'erreur, l'organisation des classes, la préparation de scénarios de test, l'utilisation d'IntelliJ et de Git, l'intégration de l'interface Swing ainsi que la structuration et la rédaction du présent compte rendu.

L'assistant a notamment aidé à relier l'exception de A7 au classpath du serveur, à distinguer une instance passée par valeur d'un objet distant et à organiser le client en point d'entrée, logique et interface console. Il a également proposé des formulations et une structure de rapport adaptées aux exigences du sujet.

J'ai vérifié les propositions en compilant le projet et en exécutant le serveur et plusieurs clients dans des processus distincts. J'ai comparé les sorties obtenues aux points de contrôle du sujet, corrigé les chemins et arguments de lancement, puis conservé uniquement les éléments cohérents avec le code et les observations. Les choix finaux, les exécutions, les validations et le dépôt du projet restent sous ma responsabilité.

13 Références

1. Rima Bachar. HAI704I TP 1 Java RMI, sujet 2026 2027, Université de Montpellier.
2. Rima Bachar. HAI704I CM 02 RMI, support de cours 2026 2027.
3. Rima Bachar. Annexe RMI workflow et aide graduée, 2026 2027.
4. Oracle. Java Platform Standard Edition 21 API Specification, packages java.rmi et java.rmi.registry.
5. Gamma Erich, Helm Richard, Johnson Ralph et Vlissides John. Design Patterns Elements of Reusable Object Oriented Software. Addison Wesley, 1994. Patron Observer.
6. RHANBOURI Omar. Dépôt GitHub du TP 1 HAI704I. https://github.com/SJOLK/RHANBOURI_Omar_HAI704I_TP1

14 Captures de validation

Les captures suivantes proviennent des exécutions successives réalisées pendant les étapes A1 à A8. Elles complètent les observations décrites dans le corps du rapport.

14 1 Étape A1 objet animal distant

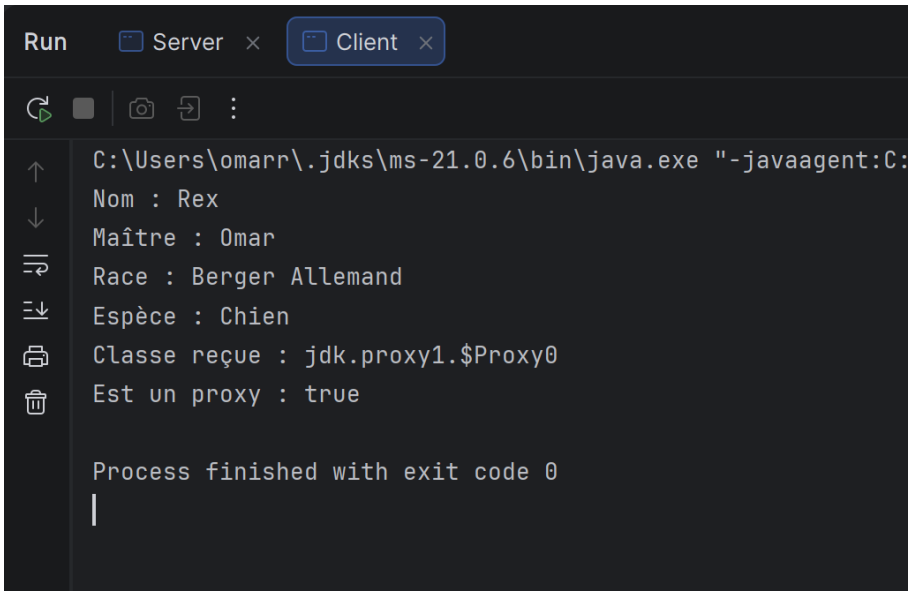


Figure 6 Le client reçoit un proxy dynamique pour Rex

14 2 Étape A2 passage de l espèce par valeur

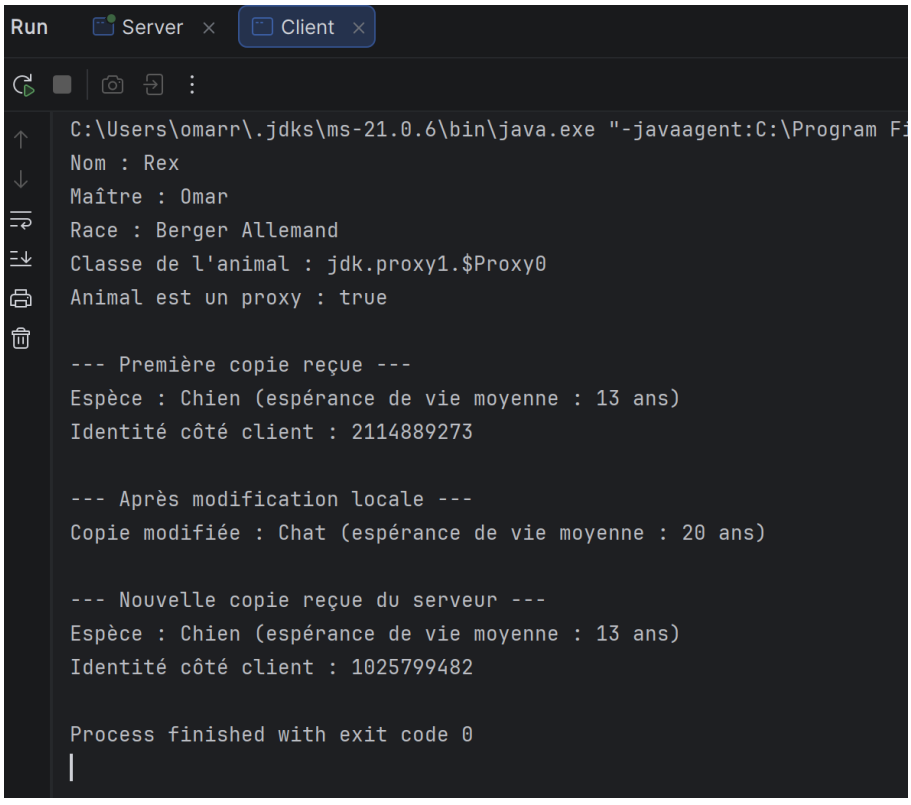


Figure 7 La modification locale ne change pas la valeur conservée sur le serveur

14 3 Étape A3 dossier de suivi distant

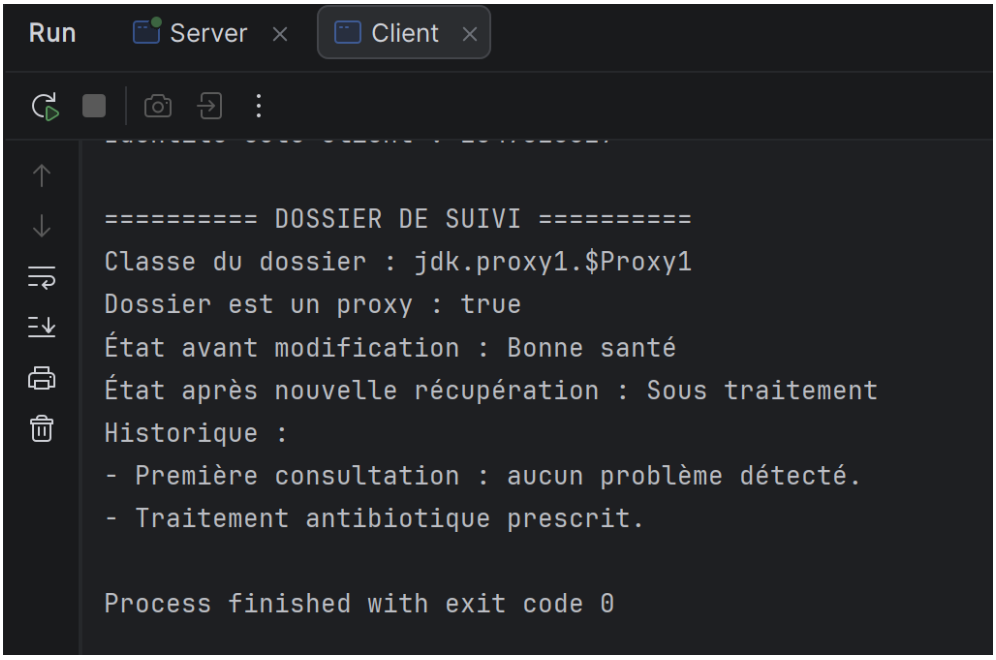


Figure 8 Le dossier est un proxy et la nouvelle lecture montre la modification

14 4 Étape A4 cabinet comme point d entrée

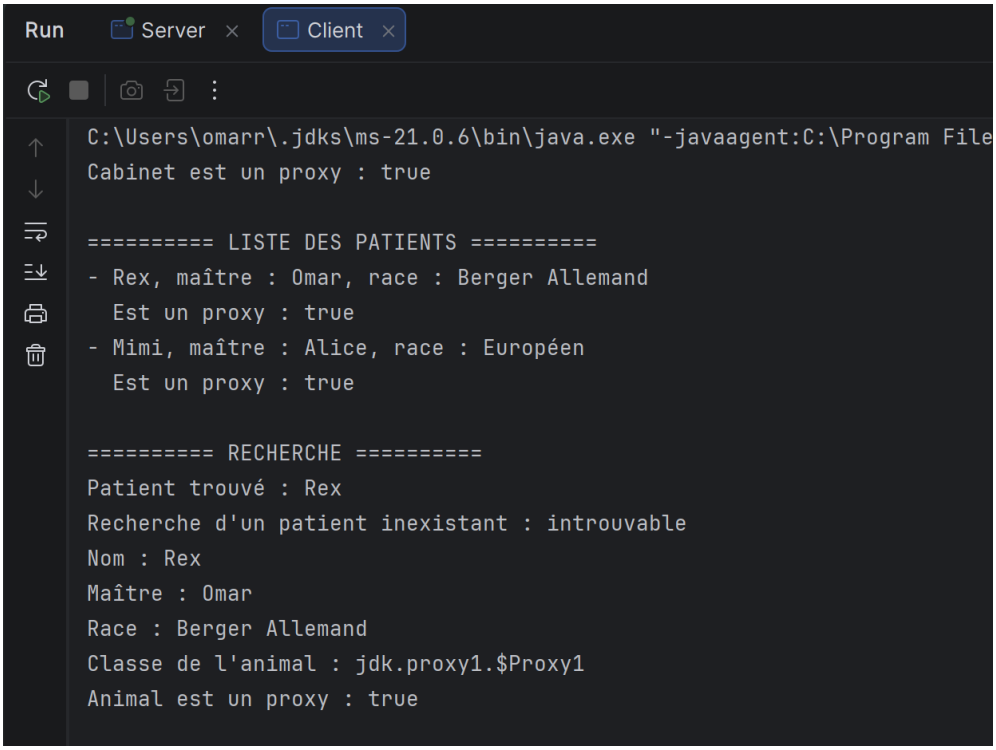


Figure 9 Liste des patients et recherche réalisées depuis le cabinet distant

14 5 Étape A5 création d un patient

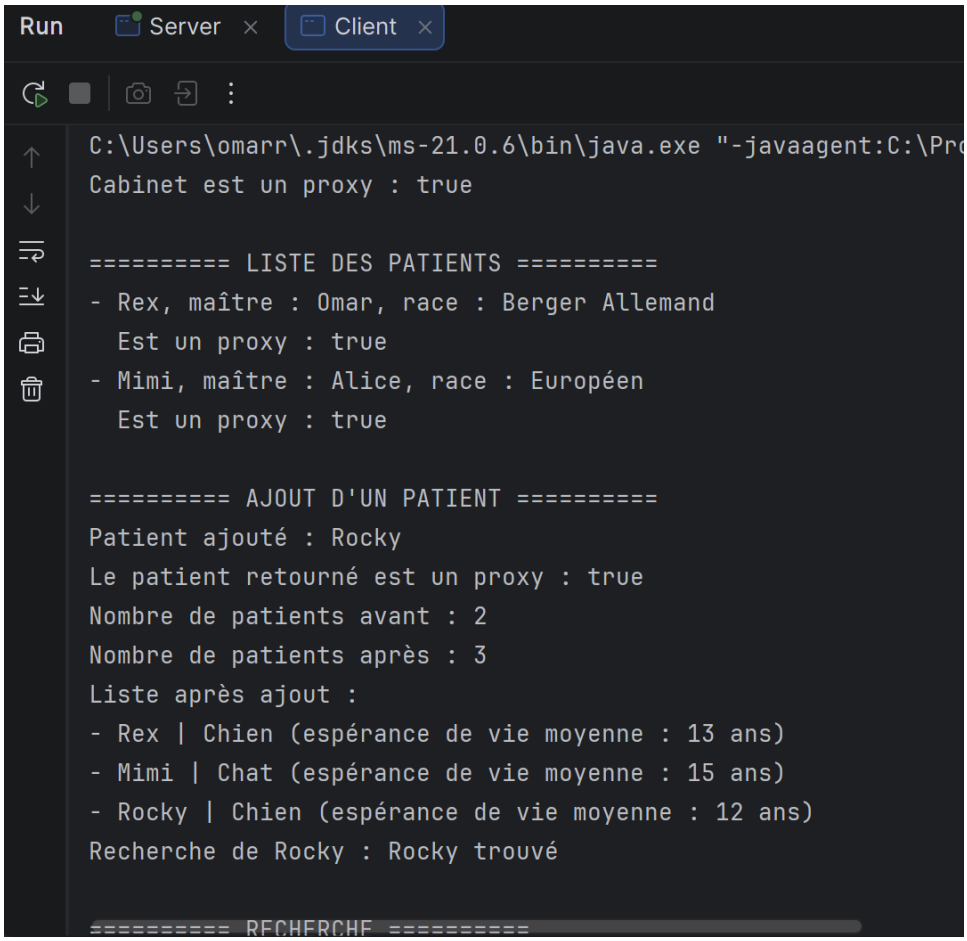


Figure 10 Rocky est créé côté serveur puis retrouvé depuis le client

14 6 Étape A6 callbacks vers deux clients

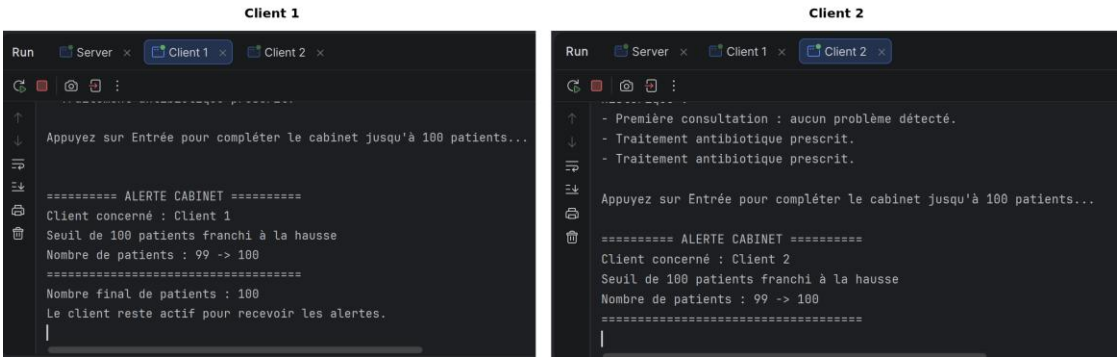


Figure 11 Les deux clients reçoivent le franchissement du seuil 100

14 7 Étape A7 sérialisation

Les figures 4 et 5 du chapitre 7 montrent l'échec lorsque Chien existe seulement dans client, puis la réussite après son déplacement dans common.

14 8 Étape A8 interface en ligne de commande

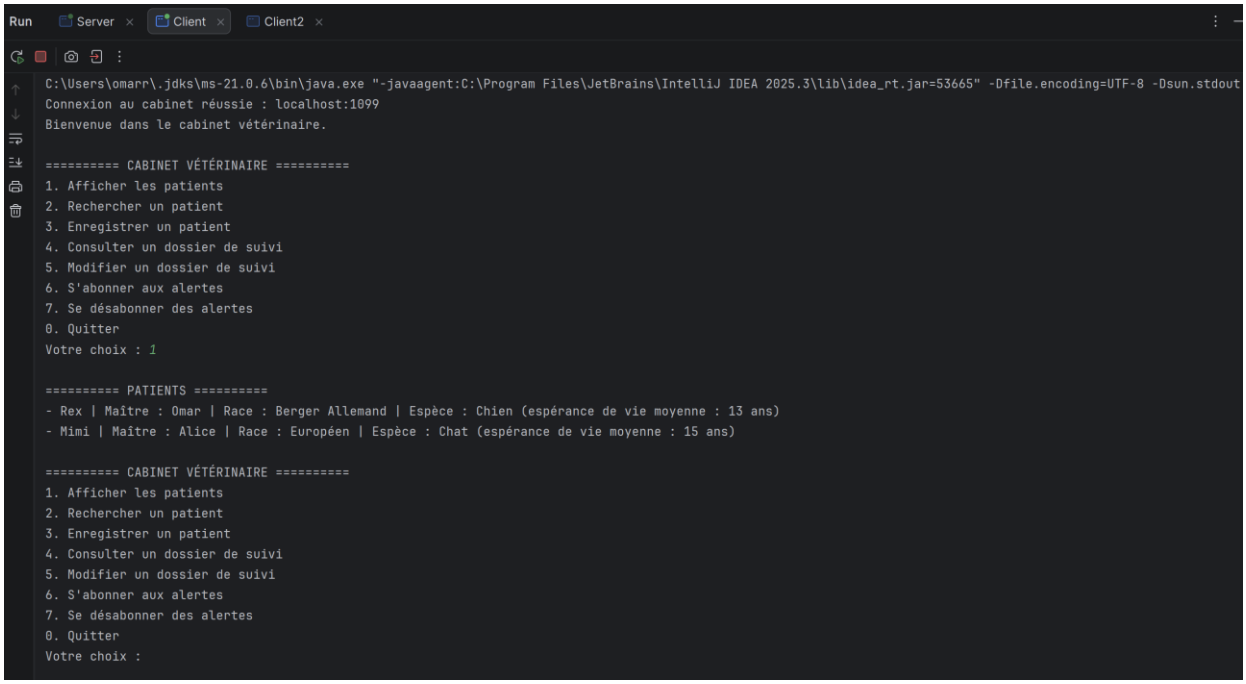


Figure 12 Menu final et affichage interactif des patients

Cette dernière exécution confirme que les opérations principales sont accessibles sans modifier le code et que la CLI s'appuie sur le même cabinet distant.